PONTIFICIA UNIVERSIDAD JAVERIANA

Facultad de Ingeniería

Departamento de Ingeniería de Sistemas

PROYECTO: SISTEMA DE CATEGORIZACIÓN METEOROLÓGICA

Curso: Sistemas Operativos

Período: Febrero - Mayo 2026

Integrantes del Grupo:

Daniel Prieto

Esteban Cantillo

Alejandro Macías

Jorge Simental

Fecha de Elaboración: 12 de mayo de 2026

TABLA DE CONTENIDOS

1. Introducción
2. Descripción General del Problema
3. Objetivos Principales
4. Desarrollo e Implementación
5. Componentes del Sistema
6. Guía de Ejecución
7. Plan de Pruebas
8. Resultados Esperados
9. Conclusiones
10. Referencias

1. INTRODUCCIÓN

Los sistemas operativos modernos requieren mecanismos robustos de sincronización y comunicación entre procesos para resolver problemas complejos de manera concurrente. Este proyecto implementa un sistema meteorológico distribuido que demuestra el uso práctico de:

• **Named pipes (FIFO)** para comunicación entre procesos
• **Semáforos POSIX** para control de acceso a recursos compartidos
• **Hilos (pthreads)** para procesamiento concurrente
• **Buffers circulares** para almacenamiento eficiente de datos

El proyecto implementa un sistema que captura, filtra y categoriza mediciones meteorológicas de tres estaciones en Bogotá: Kennedy, Usaquén y Teusaquillo.

2. DESCRIPCIÓN GENERAL DEL PROBLEMA

Contexto: El Instituto Distrital de Gestión de Riesgos y Cambio Climático necesita un sistema que centralice y procese datos meteorológicos de múltiples estaciones en tiempo real.

Problema: Las estaciones envían mediciones de forma independiente en archivos CSV. Se requiere un sistema que:
• Lea datos de múltiples fuentes simultáneamente (procesos independientes)
• Valide que los datos estén dentro de rangos aceptables
• Los organice en un archivo consolidado
• Calcule promedios y determine la categoría meteorológica final

Restricciones:
• Debe ser totalmente concurrente (sin bloqueos innecesarios)
• Evitar condiciones de carrera
• Usar IPC (Inter-Process Communication) estándar de POSIX
• Ejecutarse en Linux/Unix con soporte POSIX

3. OBJETIVOS PRINCIPALES

Objetivos Generales:
1. Implementar un sistema distribuido que demuestre el uso correcto de primitivas de sincronización en sistemas operativos.
2. Aplicar patrones de concurrencia (productor/consumidor) de forma práctica.
3. Implementar IPC mediante named pipes de forma eficiente.

Objetivos Específicos:
• Crear un agente (agenteM) que lea archivos CSV y envíe datos válidos por un pipe
• Implementar un monitor que sincronice múltiples agentes usando buffers circulares
• Usar semáforos POSIX para evitar race conditions
• Procesar datos con hilos consumidores dedicados por estación
• Generar un archivo consolidado y un reporte final de categorización

4. DESARROLLO E IMPLEMENTACIÓN

4.1 Arquitectura del Sistema
El sistema sigue una arquitectura productor/consumidor con múltiples buffers:
Productores (Agentes): Leen archivos CSV y envían mediciones válidas por el pipe.
Consumidores (Hilos): Reciben datos, los almacenan en buffers circulares y escriben en el archivo consolidado.
Monitor: Orquesta el flujo de datos usando hilos y sincronización.

4.2 Tecnologías Utilizadas
Lenguaje: C (POSIX compliant)
Compilador: GCC con flags -Wall -Wextra
IPC: Named Pipes (mkfifo)
Sincronización: Semáforos POSIX (sem_init, sem_wait, sem_post)
Threading: pthreads (pthread_create, pthread_join)
Mutex: pthread_mutex_t para acceso exclusivo

4.3 Flujo de Ejecución
1. Monitor crea el named pipe y se bloquea esperando lectores
2. Primer agente abre el pipe → Monitor se desbloquea
3. Monitor crea hilo recolector que lee del pipe
4. Recolector distribuye datos a buffers por estación
5. Consumidores escriben en archivo consolidado
6. Al cierre del último agente, monitor calcula promedios y emite categoría

5. COMPONENTES DEL SISTEMA

5.1 agenteM.c (Agente de Medición)
Responsabilidades:
• Leer archivo CSV con mediciones de sensores
• Parsear líneas en formato: ESTACION,HUMEDAD,ROCIO,PRESION,HORA
• Validar que cada medición esté dentro de rangos aceptables
• Enviar mediciones válidas por el pipe al monitor
• Detectar línea de punto (.) como fin de archivo

Parámetros de entrada:
-f archivo.csv: Ruta del archivo de sensores (obligatorio)
-t tiempo: Segundos entre lecturas (obligatorio)
-p pipeNom: Nombre del pipe nominal (obligatorio)
-k rm: Bandera para borrar archivo al terminar (opcional)

Salida en consola:
"... Agente de Medición en proceso!!!"
"[agenteM] Medición rechazada (fuera de rango): ..."
"Fin de Lectura de Sensores de la Estación [EK/ET/EU]!!!"

5.2 monitor.c (Monitor y Control de Categorización)
Responsabilidades:
• Crear el named pipe (FIFO)
• Lanzar hilo recolector que lee del pipe
• Crear buffers circulares dinámicamente por estación
• Lanzar hilos consumidores para cada estación
• Sincronizar productor/consumidor con semáforos
• Escribir archivo consolidado.csv
• Detectar datos faltantes e imprimir alarmas
• Calcular promedio global y categorizar clima final

Parámetros de entrada:
-b tamBuffer: Tamaño del buffer circular (obligatorio)
-p pipeNom: Nombre del pipe nominal (obligatorio)

Salida en consola:
"... Control de Categorización Meteorológica!!!"
"[ALARMA] Estación [X]: datos faltantes entre [hora] y [hora]..."
"Parte Meteorológico Bogotá: "[Categoría]""
"Fin del Monitor...!!!"

5.3 Estructura de Datos Compartida
Buffer circular por estación:
• Arreglo dinámico de struct Medicion
• Índices: inicio, fin, count
• Semáforos: lleno (datos disponibles), vacio (espacios libres)
• Mutex: protege los índices contra acceso concurrente
• Acumuladores: suma_hum, suma_rocio, suma_presion para promedios

6. GUÍA DE EJECUCIÓN

6.1 Estructura de Carpetas

Elemento	Tipo	Descripción
Makefile	Archivo	Reglas de compilación
agenteM.c	Código fuente	Agente de medición
monitor.c	Código fuente	Monitor concurrente
datos/kennedy.csv	Datos	Mediciones estación Kennedy (EK)
datos/usaquen.csv	Datos	Mediciones estación Usaquén (ET)
datos/sasi.csv	Datos	Mediciones estación Teusaquillo (EU)

6. GUÍA DE EJECUCIÓN

4.4. Diagramas Tecnicos del Sistema

A. Diagrama General de Arquitectura

El siguiente diagrama ilustra los componentes principales del sistema y su interaccion mediante el named pipe (FIFO), los buffers circulares y los hilos consumidores:

```
+------------------+   +------------------+   +------------------+
|   agenteM (EK)   |   |   agenteM (ET)   |   |   agenteM (EU)   |
|  Productor CSV   |   |  Productor CSV   |   |  Productor CSV   |
+--------+---------+   +--------+---------+   +--------+---------+
         |                     |                       |
         +----------+----------+----------+----------+
                              |
                        write(FIFO)
                              |
         +----------------------v------------------------------+
         |            Named Pipe (FIFO: pipeNom)               |
         +----------------------+------------------------------+
                              |
                     +---------v---------+
                     |  Hilo Recolector  |
                     |   (read + parse)  |
                     +---------+---------+
          +-------------+--------+---------+-------------+
          |                     |                       |
 +--------v--------+   +---------v------+   +----------v-------+
 | Buffer Circular |   | Buffer Circular|   | Buffer Circular  |
 |  Estacion EK    |   |  Estacion ET   |   |  Estacion EU     |
 | sem lleno/vacio |   | sem lleno/vacio|   | sem lleno/vacio  |
 | mutex + count   |   | mutex + count  |   | mutex + count    |
 +--------+--------+   +-------+--------+   +----------+-------+
          |                    |                       |
 +--------v--------+   +-------v--------+   +----------v-------+
 | Hilo Consumidor |   | Hilo Consumidor|   | Hilo Consumidor  |
 |   (EK writer)   |   |  (ET writer)   |   |  (EU writer)     |
 +--------+--------+   +-------+--------+   +----------+-------+
          |                    |                       |
          +--------------------+-----------------------+
                               |
                   +-----------v-----------+
                   |    consolidado.csv    |
                   +-----------+-----------+
                               |
                   +-----------v-----------+
                   | Calculo de promedios  |
                   | Categoria final       |
                   | (Parte Meteorologico) |
                   +-----------------------+
```

B. Diagrama Productor / Consumidor con Semaforos

Cada buffer circular es protegido por dos semaforos y un mutex, implementando el patron clasico productor/consumidor:

```
              PRODUCTOR (hilo recolector)
                        |
           +-----------v-----------+
           |  sem_wait(vacio)      |  -- Espera espacio libre
           |  mutex_lock           |  -- Exclusion mutua
           |  buffer[fin] = dato   |  -- Escribe en buffer
           |  fin = (fin+1) % tam  |  -- Avanza puntero fin
           |  count++              |  -- Incrementa contador
```

6. GUÍA DE EJECUCIÓN

```
        |  mutex_unlock         |  -- Libera mutex
        |  sem_post(lleno)      |  -- Senaliza nuevo dato
        +----------+------------+
                   |
 +------------------v------------------+
 |          BUFFER CIRCULAR            |
 |  [ ][ ][ ][D][D][D][ ][ ]           |
 |   ^ini=0        ^fin=3              |
 |   sem_vacio: espacios libres        |
 |   sem_lleno: datos disponibles      |
 |   mutex: protege ini/fin/count      |
 +------------------+------------------+
                   |
        +----------v-----------+
        |  sem_wait(lleno)     |  -- Espera dato disponible
        |  mutex_lock          |  -- Exclusion mutua
        |  dato = buffer[ini]  |  -- Lee del buffer
        |  ini=(ini+1)%tam     |  -- Avanza puntero inicio
        |  count--             |  -- Decrementa contador
        |  mutex_unlock        |  -- Libera mutex
        |  sem_post(vacio)     |  -- Senaliza espacio libre
        +-----------v----------+
                    |
          CONSUMIDOR (hilo por estacion)
```

C. Diagrama de Flujo de Ejecucion

Flujo completo desde la lectura del CSV hasta la categorizacion meteorologica final:

```
[AGENTE]                          [MONITOR / RECOLECTOR / CONSUMIDOR]
   |                                          |
[1] Leer linea CSV                            |
   |                                          |
[2] Validar rangos sensores                   |
   | Si invalido --> rechazar + log           |
   |                                          |
[3] Escribir medicion en FIFO (write)         |
   +------------------------------------------>
                                       [4] Recolector lee del FIFO
                                              |
                                       [5] Parsear campo estacion
                                       [5] sem_wait(vacio[i])
                                       [5] mutex_lock[i]
                                       [5] Insertar en buffer[i]
                                       [5] mutex_unlock[i]
                                       [5] sem_post(lleno[i])
                                              |
                                     +----------v----------+
                                     | Consumidor[i] actua |
                                     | sem_wait(lleno[i])  |
                                     | mutex_lock[i]       |
                                     | Extraer medicion    |
                                     | mutex_unlock[i]     |
                                     | sem_post(vacio[i])  |
                                     +----------+----------+
                                              |
                                     [6] Escribir en consolidado.csv
                                              |
[Agente envia '.' (fin)]                      |
   +------------------------------------------>
                                     [7] Detectar fin de agentes
                                     [7] pthread_join consumidores
                                     [7] Calcular promedios globales
                                     [7] Determinar categoria
                                     [7] Imprimir Parte Meteorologico
                                     [7] Liberar recursos + cerrar
```

6. GUÍA DE EJECUCIÓN

D. Diagrama Visual del Buffer Circular

Estructura interna del buffer circular con punteros inicio (ini) y fin, y comportamiento modular:

```
Estado inicial (buffer vacio, tam=6):
 indice:  [0]  [1]  [2]  [3]  [4]  [5]
 datos:   [ ]  [ ]  [ ]  [ ]  [ ]  [ ]
           ^ini=0 / ^fin=0    count=0

Tras 3 inserciones (A, B, C):
 indice:  [0]  [1]  [2]  [3]  [4]  [5]
 datos:   [A]  [B]  [C]  [ ]  [ ]  [ ]
           ^ini=0           ^fin=3    count=3

Tras 2 extracciones (A y B consumidos):
 indice:  [0]  [1]  [2]  [3]  [4]  [5]
 datos:   [ ]  [ ]  [C]  [ ]  [ ]  [ ]
                     ^ini=2  ^fin=3   count=1

Comportamiento circular - formulas de avance:
 fin = (fin + 1) % tam;   // Insercion
 ini = (ini + 1) % tam;   // Extraccion

Condiciones de control:
 Buffer LLENO:  count == tam  --> sem_wait(vacio) bloquea prod.
 Buffer VACIO:  count == 0    --> sem_wait(lleno) bloquea cons.
 Sobrescritura: IMPOSIBLE porque sem_vacio limita inserciones
```

6. GUÍA DE EJECUCIÓN

4.5. Explicacion Tecnica de Semaforos POSIX

Los semaforos POSIX son mecanismos de sincronizacion que controlan el acceso concurrente a recursos compartidos. En este proyecto se usan semaforos sin nombre (sem_init) por buffer circular. A continuacion se describe cada semaforo, su proposito, valor inicial y condiciones de operacion.

Tabla de Semaforos del Sistema

Semaforo	Valor Inicial	Cuando sem_wait()	Cuando sem_post()	Proposito
lleno[i]	0	Consumidor (antes de leer)	Recolector (tras insertar)	Indica datos disponibles en buffer i
vacio[i]	tamBuffer	Recolector (antes de insertar)	Consumidor (tras extraer)	Indica espacios libres en buffer i
mutex[i]	N/A (pthread_mutex)	Antes de acceder ini/fin/count	Al terminar de modificar	Exclusion mutua sobre estructura del buffer

Explicacion Conceptual

Semaforo lleno[i]: inicializado en 0 porque al iniciar no hay datos. El consumidor llama sem_wait(lleno[i]) y se bloquea hasta que el recolector deposite una medicion y llame sem_post(lleno[i]).

Semaforo vacio[i]: inicializado en tamBuffer porque el buffer comienza completamente vacio. El recolector llama sem_wait(vacio[i]) antes de insertar; si el buffer esta lleno, se bloquea hasta que el consumidor extraiga y llame sem_post(vacio[i]).

Mutex mutex[i]: garantiza que los punteros ini, fin y el contador count no sean modificados simultaneamente por el recolector y el consumidor, evitando condiciones de carrera sobre la estructura interna del buffer.

Pseudocodigo POSIX - Inicializacion y Uso

```
/* Inicializacion por buffer (una vez por estacion i) */
sem_init(&lleno[i],  0, 0);           /* Sin datos al inicio   */
sem_init(&vacio[i],  0, tamBuffer);   /* Todo el buffer vacio  */
pthread_mutex_init(&mutex[i], NULL);  /* Mutex desbloqueado    */

/* Insercion - hilo recolector */
sem_wait(&vacio[i]);                  /* Espera espacio libre  */
pthread_mutex_lock(&mutex[i]);
  buffer[i][fin[i]] = nueva_medicion;
  fin[i] = (fin[i] + 1) % tamBuffer;
  count[i]++;
pthread_mutex_unlock(&mutex[i]);
sem_post(&lleno[i]);                  /* Senaliza nuevo dato   */

/* Extraccion - hilo consumidor */
sem_wait(&lleno[i]);                  /* Espera dato disponible*/
pthread_mutex_lock(&mutex[i]);
  medicion = buffer[i][ini[i]];
  ini[i] = (ini[i] + 1) % tamBuffer;
  count[i]--;
pthread_mutex_unlock(&mutex[i]);
sem_post(&vacio[i]);                  /* Libera espacio        */
```

6. GUÍA DE EJECUCIÓN

4.6. Explicacion Detallada del Buffer Circular

El buffer circular (ring buffer) es una estructura de datos de tamano fijo que permite almacenar y recuperar elementos en orden FIFO (First In, First Out) sin desplazar datos en memoria. Se implementa un buffer por estacion meteorologica.

Componentes Internos del Buffer

Campo	Tipo	Descripcion
datos[]	struct Medicion*	Arreglo dinamico de mediciones (tamano = tamBuffer)
ini	int	Indice del proximo elemento a leer (consumidor)
fin	int	Indice donde se insertara el proximo elemento (productor)
count	int	Numero de elementos actualmente en el buffer
lleno	sem_t	Semaforo: valor = elementos disponibles para consumir
vacio	sem_t	Semaforo: valor = espacios libres para insertar
mutex	pthread_mutex_t	Protege ini, fin y count contra acceso concurrente
suma_hum	double	Acumulador de humedad para calculo de promedios
suma_rocio	double	Acumulador de punto de rocio
suma_presion	double	Acumulador de presion barometrica

Funcionamiento Modular

Los punteros ini y fin avanzan siempre con la operacion modulo: fin = (fin + 1) % tamBuffer. Esto garantiza que al llegar al final del arreglo, los indices retornan a la posicion 0, formando el comportamiento circular. La variable count permite distinguir entre buffer lleno y buffer vacio aun cuando ini == fin en ambos casos.

Pseudocodigo de la Estructura

```
typedef struct {
    Medicion *datos;            /* Arreglo circular dinamico    */
    int ini, fin, count;        /* Punteros y contador          */
    int tam;                    /* Capacidad maxima del buffer  */
    sem_t lleno, vacio;         /* Semaforos de sincronizacion  */
    pthread_mutex_t mutex;      /* Mutex de exclusion mutua     */
    double suma_hum, suma_rocio, suma_presion;
    int total;                  /* Total de mediciones validas  */
} BufferCircular;

void inicializar_buffer(BufferCircular *b, int tam) {
    b->datos = malloc(tam * sizeof(Medicion));
    b->ini = b->fin = b->count = 0;
    b->tam = tam;
    sem_init(&b->lleno, 0, 0);
    sem_init(&b->vacio, 0, tam);
    pthread_mutex_init(&b->mutex, NULL);
}
```

6. GUÍA DE EJECUCIÓN

4.7. Manejo de Errores y Robustez del Sistema

Un sistema concurrente robusto debe validar todas las operaciones criticas y manejar errores de forma apropiada sin dejar recursos sin liberar ni producir comportamientos indefinidos.

Tabla de Validaciones del Sistema

Operacion	Validacion	Accion ante fallo
mkfifo(pipeNom, 0666)	Retorno != 0 y errno != EEXIST	Mensaje de error + exit(1)
malloc() / calloc()	Retorno == NULL	Mensaje de error + exit(1)
fopen(archivo, r)	Retorno == NULL	Mensaje de error + exit(1)
open(pipeNom, O_WRONLY)	Retorno menor que 0	Mensaje de error + exit(1)
pthread_create()	Retorno != 0	Mensaje de error + exit(1)
sem_init()	Retorno != 0	Mensaje de error + exit(1)
Parametros (getopt)	Flags obligatorios faltantes	Mensaje de uso + exit(1)
Linea '.' en CSV	Deteccion de fin de agente	Cierre ordenado del pipe
Lectura de FIFO	read() retorna 0 o menor que 0	Fin de transmision, cierra consumidores
Datos fuera de rango	Validacion por sensor	Rechazo con log + continuar

Detalle de Validaciones Criticas

Validacion de mkfifo: el pipe puede existir de una ejecucion anterior. El sistema verifica errno == EEXIST para distinguir entre un error real y un FIFO pre-existente, permitiendo reutilizarlo sin abortar el proceso.

Validacion de malloc: si la asignacion de memoria falla, el proceso termina con un mensaje claro antes de intentar acceder a un puntero nulo, evitando segmentation faults.

Manejo de cierre del pipe: cuando el ultimo agente termina y cierra su extremo del FIFO, el recolector recibe EOF (read retorna 0). Esto dispara la secuencia de cierre: se envia una senal a los consumidores mediante sem_post, se llama pthread_join para esperar su terminacion, se calculan los promedios y se genera el reporte final.

Manejo de parametros invalidos: el uso de getopt permite detectar flags faltantes u opciones no reconocidas. Si un flag obligatorio (-f, -t, -p para agenteM o -b, -p para monitor) no se proporciona, el programa imprime el modo de uso correcto y termina con exit(1).

6. GUÍA DE EJECUCIÓN

4.8. Concurrencia, Sincronizacion y Mecanismos POSIX

La correcta gestion de la concurrencia es el nucleo del proyecto. A continuacion se explican los principales desafios de concurrencia y como el sistema los resuelve con primitivas POSIX.

Problema	Descripcion	Solucion implementada
Race condition	Dos hilos modifican ini/fin/count simultaneamente	pthread_mutex_t protege la seccion critica
Deadlock	Dos hilos esperan mutuamente recursos bloqueados	Orden fijo de adquisicion de locks; sem_post garantiza progreso
Busy waiting	Hilo espera activamente en bucle consumiendo CPU	sem_wait bloquea el hilo en el SO sin consumir CPU
Starvation	Un hilo nunca obtiene acceso al recurso	FIFO fair scheduling del kernel Linux para mutexes
Buffer overflow	Productor escribe en buffer lleno	sem_wait(vacio) bloquea al productor si buffer esta lleno
Buffer underflow	Consumidor lee de buffer vacio	sem_wait(lleno) bloquea al consumidor si no hay datos

Exclusion Mutua

El mutex pthread_mutex_t garantiza que en todo momento solo un hilo pueda modificar los campos ini, fin y count del buffer. Sin mutex, dos hilos podrian leer el mismo valor de fin, calcular la misma posicion de insercion y sobrescribirse mutuamente.

Ausencia de Deadlocks

El diseno evita deadlocks por construccion: el orden de adquisicion es siempre sem_wait -> mutex_lock -> operacion -> mutex_unlock -> sem_post, nunca a la inversa. Los semaforos siempre tienen un sem_post correspondiente, garantizando que los hilos bloqueados eventualmente sean despertados.

Terminacion Ordenada con pthread_join

El monitor llama pthread_join() sobre cada hilo consumidor despues de detectar el fin del FIFO. Esto garantiza que todos los consumidores terminen de procesar sus datos pendientes antes de que el monitor calcule los promedios finales, evitando condiciones de carrera en los acumuladores.

6. GUÍA DE EJECUCIÓN

4.9. Validacion de Datos de Sensores

El agente agenteM valida cada medicion antes de enviarla por el pipe. Los rangos son coherentes con condiciones meteorologicas reales para Bogota D.C. (altitud aprox. 2600 m.s.n.m.).

Tabla de Rangos Validos por Sensor

Sensor	Variable	Rango Valido	Unidad	Accion fuera de rango
Humedad relativa	HUMEDAD	0 - 100	%	Rechazado con log en consola
Punto de rocio	ROCIO	-20 a 40	C	Rechazado con log en consola
Presion barometrica	PRESION	600 - 1100	hPa	Rechazado con log en consola

Nota: la presion en Bogota ronda los 750 hPa por la altitud. El rango 600-1100 hPa cubre condiciones extremas sin rechazar mediciones validas de alta altitud.

Logica de Validacion en agenteM

```
/* Validacion de rangos en agenteM.c */
int es_valido(float humedad, float rocio, float presion) {
    if (humedad  <  0   || humedad  > 100)  return 0;
    if (rocio    < -20  || rocio    >  40)  return 0;
    if (presion  < 600  || presion  > 1100) return 0;
    return 1;
}
/* Mediciones invalidas se registran y se descartan.  */
/* El agente NO termina: sigue procesando el CSV.    */
```

6. GUÍA DE EJECUCIÓN

4.10. Categorizacion Meteorologica

Una vez que todos los agentes han terminado y el monitor ha procesado todas las mediciones validas, se procede al calculo de promedios globales y a la determinacion de la categoria meteorologica final para Bogota.

Calculo de Promedios

Cada buffer circular mantiene acumuladores (suma_hum, suma_rocio, suma_presion) y un contador total de mediciones validas procesadas. Al finalizar, el monitor calcula el promedio global:

```
/* Promedios globales al finalizar todos los agentes */
double prom_hum     = suma_hum_total     / total_mediciones;
double prom_rocio   = suma_rocio_total   / total_mediciones;
double prom_presion = suma_presion_total / total_mediciones;
```

Criterios de Clasificacion Meteorologica

Categoria	Condicion principal	Descripcion
Tormenta	Humedad > 90% y Presion < 720 hPa	Alta humedad con baja presion: frentes tormentosos
Lluvia	Humedad > 80% y Presion < 750 hPa	Condiciones lluviosas tipicas de Bogota
Nublado	Humedad > 70% o Presion < 760 hPa	Cielo cubierto sin precipitacion inmediata
Parcialmente nublado	Humedad 50-70% y Presion 760-780 hPa	Condiciones mixtas con intervalos nublados
Despejado	Humedad < 50% y Presion > 780 hPa	Dia soleado con baja humedad relativa

La categorizacion evalua los promedios globales de todas las estaciones combinadas, priorizando condiciones de mayor severidad. El resultado se imprime como el 'Parte Meteorologico Bogota'.

6. GUÍA DE EJECUCIÓN

7.2. Pruebas Adicionales de Concurrencia y Sincronizacion

Las siguientes pruebas complementan el plan de pruebas existente. Se enfocan en escenarios de concurrencia real, manejo de errores y comportamiento de semaforos bajo condiciones limite.

Prueba A - Verificacion de Exclusion Mutua (Race Condition Test)

Objetivo: Confirmar que el mutex protege correctamente los indices del buffer bajo alta concurrencia (retardo = 0 fuerza maximo solapamiento entre hilos).

```
# Terminal 1:
$ ./monitor -b 2 -p pipeNom

# Terminal 2 (simultaneamente):
$ ./agenteM -f datos/kennedy.csv -t 0 -p pipeNom

# Terminal 3 (simultaneamente):
$ ./agenteM -f datos/usaquen.csv -t 0 -p pipeNom

# Terminal 4 (simultaneamente):
$ ./agenteM -f datos/sasi.csv    -t 0 -p pipeNom
```

Resultado esperado: consolidado.csv debe contener exactamente el numero de lineas validas sin duplicados ni datos corruptos. El count final de cada buffer debe ser 0 al terminar.

Captura requerida: salida del monitor mostrando la categoria final sin errores de sincronizacion + resultado de: $ wc -l consolidado.csv

Prueba B - Bloqueo de Semaforos con Buffer de Tamano 1

Objetivo: Verificar que sem_wait bloquea correctamente al productor cuando el buffer esta lleno. Con tamano 1, cada insercion fuerza un bloqueo.

```
# Terminal 1:
$ ./monitor -b 1 -p pipeNom

# Terminal 2:
$ ./agenteM -f datos/kennedy.csv -t 2 -p pipeNom
# (retardo t=2 para observar el intercalado productor-consumidor)
```

Resultado esperado: el agente envia una medicion y queda bloqueado en sem_wait(vacio) hasta que el consumidor extrae. La salida muestra procesamiento secuencial alternado.

Captura requerida: consola del monitor mostrando procesamiento secuencial de mediciones EK con el reporte final correcto.

Prueba C - Manejo de Error: Pipe Inexistente

Objetivo: Verificar que agenteM falla ordenadamente si se inicia antes que el monitor (el FIFO no existe aun).

```
# Sin iniciar monitor primero:
$ ./agenteM -f datos/kennedy.csv -t 1 -p pipeNoExiste
# Resultado esperado: mensaje de error + exit != 0
```

Resultado esperado: mensaje de error indicando que el pipe no puede abrirse. Sin crash ni comportamiento indefinido.

Captura requerida: salida de error del agente mostrando el mensaje de fallo de apertura.

Prueba D - Terminacion Ordenada con pthread_join

Objetivo: Confirmar que el monitor espera a todos los hilos consumidores antes de calcular los promedios finales, garantizando consistencia de datos.

6. GUÍA DE EJECUCIÓN

```
# Ejecutar sistema completo con retardo alto (t=3):
# Terminal 1: $ ./monitor -b 4 -p pipeNom
# Terminal 2: $ ./agenteM -f datos/kennedy.csv -t 3 -p pipeNom
# Terminal 3: $ ./agenteM -f datos/usaquen.csv -t 3 -p pipeNom
# Terminal 4: $ ./agenteM -f datos/sasi.csv    -t 3 -p pipeNom

# Al terminar agentes, verificar que el monitor imprime:
# 'Fin del Monitor...!!!'
# SOLO DESPUES de que aparezcan las 3 lineas 'Fin de Lectura'
```

Resultado esperado: el reporte final aparece unicamente despues de que todos los agentes hayan terminado, demostrando que pthread_join sincroniza correctamente la terminacion.

Captura requerida: ambas consolas (agentes y monitor) mostrando el orden correcto de terminacion (agentes primero, luego el reporte del monitor).

6.2 Pasos de Compilación
Comando: **$ make**
Esto ejecuta:
• gcc -Wall -Wextra -g -o agenteM agenteM.c
• gcc -Wall -Wextra -g -pthread -o monitor monitor.c

6.3 Ejecución en 4 Terminales

Terminal 1 — Monitor (PRIMERO):
$ ./monitor -b 4 -p pipeNom
(Se queda esperando que se conecten agentes)

Terminal 2 — Agente Kennedy:
$ ./agenteM -f datos/kennedy.csv -t 1 -p pipeNom
(Envía mediciones estación EK)

Terminal 3 — Agente Usaquén:
$ ./agenteM -f datos/usaquen.csv -t 1 -p pipeNom
(Envía mediciones estación ET)

Terminal 4 — Agente Teusaquillo:
$ ./agenteM -f datos/sasi.csv -t 1 -p pipeNom
(Envía mediciones estación EU)

Cuando termina el último agente, el monitor imprime el reporte final y cierra.

6.4 Verificación de Resultados
$ cat consolidado.csv
(Muestra todas las mediciones válidas procesadas)

$ wc -l consolidado.csv
(Deberías ver ~24 líneas, dependiendo de rechazos)

6.5 Limpieza
$ make clean
(Borra binarios, consolidado.csv y pipeNom)

7. PLAN DE PRUEBAS

Caso 1 — Operación Normal
Entrada: 3 agentes con datos válidos e inválidos
Salida esperada: consolidado.csv con ~24 líneas, reporte final "Nublado"

Caso 2 — Flags en Orden Distinto
Comando: $ ./agenteM -p pipeNom -t 1 -f datos/kennedy.csv
Esperado: Funciona igual (getopt maneja cualquier orden)

Caso 3 — Buffer Muy Pequeño (b=1)
Comando: $ ./monitor -b 1 -p pipeNom
Esperado: Funciona (solo fuerza más bloqueos/desbloqueos)

Caso 4 — Con Flag -k rm
Comando: $ ./agenteM -f datos/kennedy.csv -t 0 -p pipeNom -k rm
Esperado: El archivo se borra al terminar

Caso 5 — Un Solo Agente
Comando: $ ./monitor -b 4 -p pipeNom; ./agenteM -f datos/kennedy.csv -t 1 -p pipeNom
Esperado: Funciona (crea solo una estación en el monitor)

Caso 6 — Datos Todos Inválidos
Entrada: Archivo con mediciones fuera de rango
Salida esperada: consolidado.csv vacío o con pocas líneas

Caso 7 — Detección de Horas Faltantes
Entrada: Datos con 2+ horas de diferencia sin registros
Salida esperada: Aparece [ALARMA] en consola del monitor

8. RESULTADOS ESPERADOS

Salida en Consola del Monitor:

```
... Control de Categorización Meteorológica!!! [ALARMA] Estación EK: datos faltantes entre
13:00:00 y 14:00:00... Parte Meteorológico Bogotá: "Nublado" Fin del Monitor...!!!
```

Contenido de consolidado.csv:

```
EK,92,10,748,08:00:00 EK,95,11,745,09:00:00 EK,91,10,747,10:00:00 EK,88,9,751,11:00:00
EK,93,11,746,12:00:00 EK,90,10,749,14:00:00 ET,85,9,751,08:00:00 ET,82,8,753,09:00:00
ET,90,10,749,10:00:00 ... (más líneas)
```

9. CONCLUSIONES

Este proyecto demuestra la implementación exitosa de un sistema distribuido y concurrente que utiliza primitivas fundamentales de sincronización en sistemas operativos:

Logros técnicos:
✓ Comunicación entre procesos mediante named pipes (FIFO)
✓ Sincronización sin condiciones de carrera usando semáforos POSIX
✓ Patrón productor/consumidor con buffers circulares
✓ Procesamiento concurrente con múltiples hilos
✓ Validación y manejo robusto de errores

Aprendizajes obtenidos:
• Comprensión profunda de IPC (Inter-Process Communication)
• Manejo correcto de sincronización y exclusión mutua
• Diseño de sistemas concurrentes sin deadlocks
• Implementación de patrones de concurrencia clásicos
• Programación POSIX de bajo nivel en C

El sistema está listo para procesamiento de datos en tiempo real de múltiples sensores meteorológicos distribuidos, con garantías de consistencia y sin pérdida de datos.

10. REFERENCIAS

[1] Stevens, W. R., & Rago, S. A. (2013). *Advanced Programming in the UNIX Environment* (3rd ed.). Addison-Wesley Professional.

[2] Butenhof, D. R. (1997). *Programming with POSIX Threads*. Addison-Wesley Professional.

[3] POSIX Specification. (2018). *IEEE Std 1003.1-2017: The Open Group Base Specifications Issue 7*. Retrieved from http://pubs.opengroup.org/onlinepubs/9699919799/

[4] Linux man pages online. (2024). Retrieved from https://man7.org/linux/man-pages/

[5] Lecture notes: Sistemas Operativos 2026-30. Pontificia Universidad Javeriana.